

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – Pencarian Nilai Max/Min



NAMA : Wahid Satrio Aji

NIM : 109082500005

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam ilmu komputer dan pemrograman, pemrosesan data numerik merupakan operasi fundamental yang sering dilakukan, salah satunya adalah pencarian nilai ekstrem, yaitu nilai maksimum (terbesar) dan minimum (terkecil), serta perhitungan nilai rata-rata (mean). Untuk menangani sekumpulan data dalam bahasa pemrograman Go, digunakan struktur data seperti array yang memiliki kapasitas memori statis (tetap), maupun slice yang bersifat dinamis dan panjangnya dapat disesuaikan saat eksekusi program. Selain tipe data dasar, pencarian nilai ekstrem juga dapat diterapkan pada kumpulan data terstruktur menggunakan tipe data bentukan atau struct, yang memungkinkan programmer menggabungkan beberapa atribut berbeda (seperti nama, NIM, dan IPK) ke dalam satu entitas kesatuan.

Algoritma dasar yang digunakan untuk mencari nilai maksimum dan minimum dalam sekumpulan data yang tidak terurut adalah pencarian sekuensial (Linear Search). Algoritma ini bekerja dengan cara menetapkan elemen pertama dari kumpulan data sebagai nilai patokan sementara (basis pencarian), lalu melakukan iterasi (perulangan) untuk membandingkan elemen-elemen berikutnya satu per satu hingga akhir data. Apabila ditemukan elemen yang nilainya melampaui batas ekstrem patokan tersebut, maka nilai patokan akan diperbarui. Pendekatan ini memiliki kompleksitas waktu $O(N)$, yang berarti waktu eksekusinya berbanding lurus dengan jumlah total data yang diproses.

Untuk mengoptimalkan struktur penulisan kode, logika pencarian ini umumnya dipecah ke dalam subprogram atau fungsi yang terpisah dari program utama. Modularitas ini membuat kode menjadi lebih rapi, mudah dibaca, dan dapat digunakan kembali (reusable). Secara bawaan (default), fungsi di dalam bahasa Go menggunakan mekanisme pass-by-value yang hanya mengirimkan salinan data. Namun, untuk kasus di mana fungsi perlu mengubah lebih dari satu nilai secara bersamaan di dalam memori utama, programmer dapat memanfaatkan mekanisme pass-by-reference menggunakan pointer (simbol asteris * dan ampersand &). Melalui proses dereferencing, fungsi dapat langsung memanipulasi alamat memori dari variabel terkait tanpa harus mengembalikan nilai melalui return.

B. Guided

1. Minmax array

```
package main

import "fmt"

func nilaiTerbesar(array []int) (int, int) {
    var idx int
    var nilaiTerbesar int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] > nilaiTerbesar {
            nilaiTerbesar = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerbesar, idxDitemukan
}

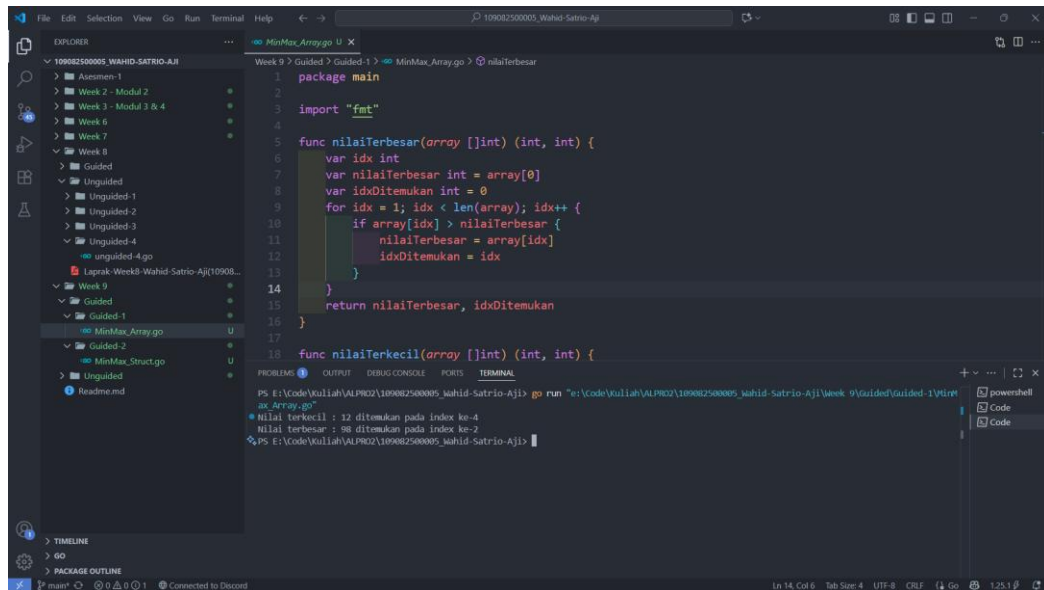
func nilaiTerkecil(array []int) (int, int) {
    var idx int
    var nilaiTerkecil int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] < nilaiTerkecil {
            nilaiTerkecil = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerkecil, idxDitemukan
}

func main() {
    var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

    var nilaiMaks, idxNilaiMaks int =
    nilaiTerbesar(arrAngka[:])
    var nilaiMin, idxNilaiMin int =
    nilaiTerkecil(arrAngka[:])

    fmt.Printf("Nilai terkecil : %d ditemukan pada
index ke-%d", nilaiMin, idxNilaiMin)
    fmt.Println()
    fmt.Printf("Nilai terbesar : %d ditemukan pada
index ke-%d", nilaiMaks, idxNilaiMaks)
}
```

Output :



```
1 package main
2
3 import "fmt"
4
5 func nilaiTerbesar(array []int) (int, int) {
6     var idx int
7     var nilaiTerbesar int = array[0]
8     var idxDitemukan int = 0
9     for idx = 1; idx < len(array); idx++ {
10         if array[idx] > nilaiTerbesar {
11             nilaiTerbesar = array[idx]
12             idxDitemukan = idx
13         }
14     }
15     return nilaiTerbesar, idxDitemukan
16 }
17
18 func nilaiTerkecil(array []int) (int, int) {
19     var idx int
20     var nilaiTerkecil int = array[0]
21     var idxDitemukan int = 0
22     for idx = 1; idx < len(array); idx++ {
23         if array[idx] < nilaiTerkecil {
24             nilaiTerkecil = array[idx]
25             idxDitemukan = idx
26         }
27     }
28     return nilaiTerkecil, idxDitemukan
29 }
30
31 func main() {
32     array := []int{12, 45, 98, 23, 67}
33     nilaiTerbesar, idxDitemukan := nilaiTerbesar(array)
34     nilaiTerkecil, idxDitemukan := nilaiTerkecil(array)
35     fmt.Println("Nilai terbesar : ", nilaiTerbesar, "ditemukan pada index ke-", idxDitemukan)
36     fmt.Println("Nilai terkecil : ", nilaiTerkecil, "ditemukan pada index ke-", idxDitemukan)
37 }
```

PS E:\Code\Guliah\ALPRO2\109082500005_wahid-Satrio-Aji> go run "e:\Code\Guliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 9\Guided\Guided-1\MinMax_Array.go"

Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS E:\Code\Guliah\ALPRO2\109082500005_wahid-Satrio-Aji>

Penjelasan :

Program ini bertujuan untuk mencari nilai terbesar dan terkecil beserta posisi indeksinya di dalam sebuah himpunan data bilangan bulat. Program menggunakan pendekatan modular dengan memisahkan logika pencarian ke dalam dua fungsi utama, yakni nilaiTerbesar dan nilaiTerkecil. Pada kedua fungsi tersebut, program menginisialisasi pencarian dengan menjadikan elemen pertama dari array sebagai nilai patokan awal (basis pencarian), dan menetapkan indeks penemuannya di angka nol. Selanjutnya, program melakukan iterasi perulangan mulai dari indeks pertama hingga elemen terakhir untuk membandingkan setiap data dengan nilai patokan. Apabila ditemukan elemen yang nilainya lebih besar (pada fungsi nilaiTerbesar) atau lebih kecil (pada fungsi nilaiTerkecil) dari patokan saat ini, maka variabel nilai ekstrem dan variabel pelacak indeks akan langsung diperbarui secara bersamaan. Pada blok fungsi utama (main), sebuah array statis berisi lima angka diinisialisasi lalu dikirim ke dalam fungsi-fungsi tersebut melalui mekanisme slice, kemudian fungsi akan mengembalikan dua nilai sekaligus (multiple return values) untuk dicetak ke layar guna menunjukkan angka ekstrem beserta lokasi persis indeksinya.

2. Minmax struct

```
package main

import "fmt"
```

```

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("=====")
    }
}

```

```

fmt.Println()
for j = 0; j < len(arrMhs); j++ {
    fmt.Println("DATA MAHASISWA INDEKS KE-", j)
    fmt.Println("Nama : ", arrMhs[j].nama)
    fmt.Println("NIM : ", arrMhs[j].nim)
    fmt.Println("IPK : ", arrMhs[j].IPK)
    fmt.Println("=====")
}
fmt.Println()

var mhsMaks, mhsMin mahasiswa
var idxMaks, idxMin int

mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
mhsMin, idxMin = IPKTerkecil(arrMhs[:])

fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim, idxMaks)
fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim, idxMin)
}

```

Output :

```

Week 9 > Guided > Guided-2 > MinMax_Struct.go > IPKTerbesar
13 func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
14     var idxDitemukan int = 0
15     var i int
16
17     for i = 1; i < len(array); i++ {
18         if array[i].IPK > mhsTerbesar.IPK {
19             mhsTerbesar = array[i]
20             idxDitemukan = i
21         }
22     }
23 }
24
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
PS E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji> go run "e:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 9\Guided\Guided-2\MinMax_Struct.go"
Masukkan IPK : 3.9
=====
DATA MAHASISWA INDEKS KE- 0
Nama : Wahid
NIM : 109082500005
IPK : 4
=====
DATA MAHASISWA INDEKS KE- 1
Nama : Syahrul
NIM : 10908256723
IPK : 3.8
=====
DATA MAHASISWA INDEKS KE- 2
Nama : Dika
NIM : 10908241921
IPK : 3.9
=====
=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 4.00, atas nama Wahid dengan NIM 109082500005, ditemukan pada indeks ke-0IPK Terkecil : 3.80, atas nama Syahrul dengan NIM 10908256723, ditemukan pada indeks ke-1
PS E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji>

```

Penjelasan :

Program ini merupakan eskalasi dari algoritma pencarian nilai ekstrem yang kini diterapkan pada tipe data terstruktur (struct). Langkah pertama yang dilakukan adalah mendefinisikan tipe data struct bernama mahasiswa yang membungkus entitas nama, NIM, dan IPK ke dalam satu kesatuan, dilanjutkan dengan pembuatan array khusus untuk menampung tiga objek mahasiswa tersebut. Logika komparasinya dienkapsulasi ke dalam fungsi IPKTerbesar dan IPKTerkecil yang menerima parameter berupa slice dari struct mahasiswa. Konsep pencariannya serupa dengan array biasa, di mana data mahasiswa pada indeks awal dijadikan referensi, namun pada proses iterasinya, program secara spesifik membandingkan atribut .IPK dari setiap entitas. Jika ditemukan mahasiswa dengan nilai IPK yang lebih ekstrem dari referensi, program tidak hanya menggeser nilai IPK-nya saja, melainkan langsung menimpa (overwrite) seluruh objek struct tersebut ke dalam variabel penyimpan beserta indeks lokasinya. Pada fungsi eksekusi utama, program memfasilitasi interaksi dua arah dengan melakukan perulangan untuk meminta masukan (input) data mahasiswa dari pengguna, mencetak ulang data yang berhasil disimpan, dan memanggil fungsi pencarian untuk menampilkan identitas lengkap mahasiswa peraih IPK tertinggi dan terendah secara komprehensif.

C. Unguided

1. Soal Unguided 1

- 1) Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var n int
    var beratKelinci [1000]float64

    fmt.Scan(&n)

    if n <= 0 {
        return
    }

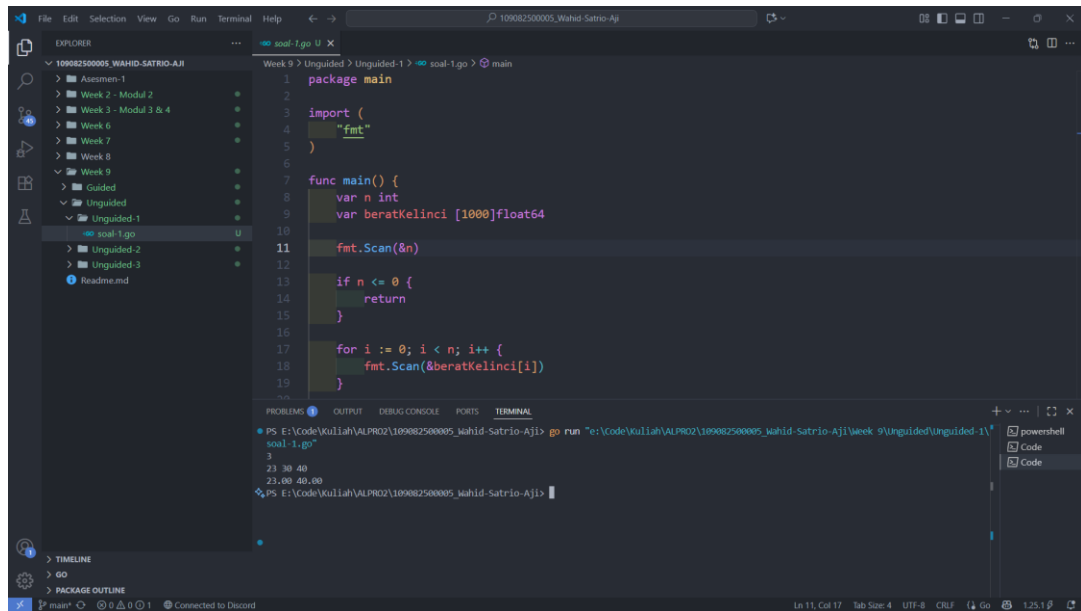
    for i := 0; i < n; i++ {
        fmt.Scan(&beratKelinci[i])
    }

    min := beratKelinci[0]
    max := beratKelinci[0]

    for i := 1; i < n; i++ {
        if beratKelinci[i] < min {
            min = beratKelinci[i]
        }
        if beratKelinci[i] > max {
            max = beratKelinci[i]
        }
    }

    fmt.Printf("%.2f %.2f\n", min, max)
}
```

Output :



The screenshot shows a Visual Studio Code editor with a Go file named `soal-1.go`. The code implements a linear search algorithm to find the minimum and maximum values in an array of 1000 float64 elements. The program includes a package declaration, imports, variable declarations for `n` and the array `beratKelinci`, a `main` function, and a loop that iterates through the array to find the min and max values. The terminal at the bottom shows the command `go run "E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 9\Unguided\Unguided 1\soal-1.go"` and its output, which displays the min and max values of the array.

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func main() {
8     var n int
9     var beratKelinci [1000]float64
10
11     fmt.Scan(&n)
12
13     if n <= 0 {
14         return
15     }
16
17     for i := 0; i < n; i++ {
18         fmt.Scan(&beratKelinci[i])
19     }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

PS E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji> go run "E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 9\Unguided\Unguided 1\soal-1.go"

3

23.30 40

23.00 40.00

PS E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji>

Penjelasan :

Program ini menggunakan pencarian linear dengan kompleksitas waktu $O(N)$ untuk menemukan nilai minimum dan maksimum dalam data. Pada tahap awal, program menginisialisasi memori dengan mendeklarasikan sebuah array dengan nama `beratKelinci` yang memiliki ukuran tetap 1000 elemen bertipe data `float64`, sehingga mengalokasikan blok memori secara berurutan. Setelah itu, program melakukan menggunakan edge case melalui pengecekan kondisi untuk memastikan jumlah kelinci lebih dari nol, jika tidak, program akan langsung berhenti untuk mencegah terjadinya *error* di luar batas saat mengakses elemen array. Sebagai basis pencarian awal, variabel nilai minimum dan maksimum diinisialisasi langsung menggunakan nilai dari elemen pertama array. Kemudian, program menjalankan proses iterasi dari indeks kedua hingga akhir, secara bertahap membandingkan setiap elemen dengan nilai patokan yang tersimpan, lalu menimpa (*overwrite*) variabel minimum atau maksimum apabila ditemukan nilai elemen baru.

2. Soal Unguided 2

2) Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var x, y int
    var beratIkan [1000]float64

    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Scan(&beratIkan[i])
    }

    var totalWadah []float64
    var beratSementara float64
    var hitungIkan int

    for i := 0; i < x; i++ {
        beratSementara += beratIkan[i]
        hitungIkan++

        if hitungIkan == y || i == x-1 {
            totalWadah = append(totalWadah,
            beratSementara)
            beratSementara = 0
            hitungIkan = 0
        }
    }
}
```

```

var totalSemuaWadah float64

for i := 0; i < len(totalWadah); i++ {
    fmt.Printf("%.2f", totalWadah[i])
    if i < len(totalWadah)-1 {
        fmt.Print(" ")
    }
    totalSemuaWadah += totalWadah[i]
}
fmt.Println()

rataRata := totalSemuaWadah /
float64(len(totalWadah))
fmt.Printf("%.2f\n", rataRata)
}

```

Output :

The screenshot shows a VS Code editor with a Go file named `soal-2.go` open. The code in the editor is as follows:

```

package main

import (
    "fmt"
)

func main() {
    var x, y int
    var beratikan [1000]float64

    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Scan(&beratikan[i])
    }

    var totalWadah []float64
    var beratSementara float64
    var hitungikan int

```

The terminal output shows the execution of the program. It prompts for input and displays the results of the calculations. The output is as follows:

```

PS E:\Code\Kuliah\ALPRO2\109082500005_Mahid-Satrio-Aji> go run "e:\Code\Kuliah\ALPRO2\109082500005_Mahid-Satrio-Aji\Week 9\Unguided\Unguided-2\soal-2.go"
4 3
2.5 3.5 4.0 1.0
10.00 1.00
5.50
PS E:\Code\Kuliah\ALPRO2\109082500005_Mahid-Satrio-Aji> go run "e:\Code\Kuliah\ALPRO2\109082500005_Mahid-Satrio-Aji\Week 9\Unguided\Unguided-2\soal-2.go"
5 2
1 2 3 4 5
3.00 7.00 5.00
5.00
PS E:\Code\Kuliah\ALPRO2\109082500005_Mahid-Satrio-Aji>

```

Penjelasan :

Program kedua menggabungkan penggunaan array untuk menampung data input mentah dan slice untuk menyimpan hasil komputasi yang panjang elemennya bergantung pada kalkulasi saat eksekusi. Berbeda dengan array,

slice bernama `totalWadah` bersifat fleksibel dan ukurannya bisa terus bertambah. Logika pengelompokan (batching) dijalankan di dalam sebuah perulangan yang memanfaatkan dua variable pembantu, yakni sebuah akumulator untuk terus menjumlahkan berat ikan dan sebuah penghitung (counter) untuk melacak jumlah isi wadah saat ini. Program menerapkan kondisi pemotongan (cut-off) yang ketika penghitung telah menyentuh batas kapasitas maksimal wadah atau ketika iterasi telah mencapai ikan terakhir di dalam array, perhitungan berat sementara tersebut akan disisipkan ke dalam slice menggunakan fungsi bawaan Go, dan kedua variabel pembantunya akan langsung di-reset ke nol untuk perulangan selanjutnya. Pada tahapan akhir, program melakukan iterasi kedua untuk membaca setiap elemen di dalam slice, mencetak berat masing-masing wadah ke layar, sekaligus menjumlahkan seluruh berat tersebut yang kemudian tipe datanya diubah (casting) menjadi `float64` agar bisa dibagi secara presisi dengan total panjang slice guna mendapatkan nilai rata-rata keseluruhan.

3. Soal Unguided 3

- 3) Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Buatlah program dengan spesifikasi subprogram sebagai berikut:

Jawaban :

```
package main

import (
    "fmt"
)

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]
```

```

        for i := 1; i < n; i++ {
            if arrBerat[i] < *bMin {
                *bMin = arrBerat[i]
            }
            if arrBerat[i] > *bMax {
                *bMax = arrBerat[i]
            }
        }
    }

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var n int
    var dataBalita arrBalita

    fmt.Print("Masukan banyak data berat balita: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&dataBalita[i])
    }

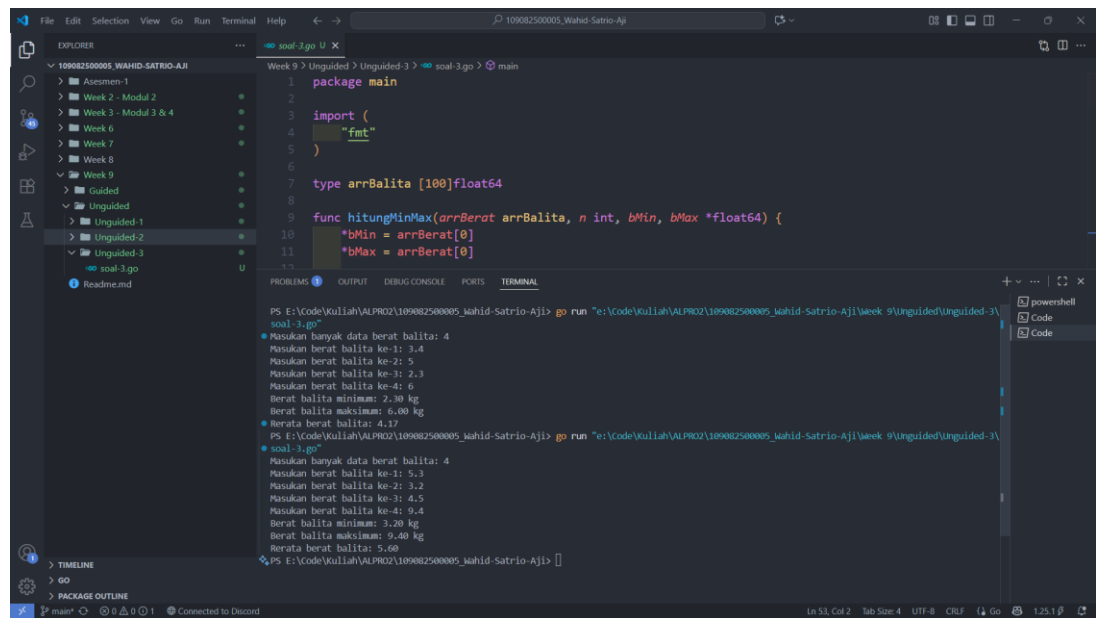
    var min, max float64

    if n > 0 {
        hitungMinMax(dataBalita, n, &min, &max)
        rataRata := rerata(dataBalita, n)

        fmt.Printf("Berat balita minimum: %.2f kg\n",
min)
        fmt.Printf("Berat balita maksimum: %.2f kg\n",
max)
        fmt.Printf("Rerata berat balita: %.2f\n",
rataRata)
    }
}

```

Output :



```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 type arrBalita [100]float64
8
9 func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
10     *bMin = arrBerat[0]
11     *bMax = arrBerat[0]
12 }
13
14 func main() {
15     n := 4
16     arrBerat := make([]float64, n)
17     for i := 0; i < n; i++ {
18         fmt.Println("Masukan berat balita ke-", i+1)
19         arrBerat[i] = 3.4
20     }
21     bMin := 0.0
22     bMax := 0.0
23     hitungMinMax(arrBerat, n, &bMin, &bMax)
24     fmt.Println("Berat balita minimum: ", bMin, "kg")
25     fmt.Println("Berat balita maksimum: ", bMax, "kg")
26     fmt.Println("Rata-rata berat balita: ", bMin+bMax, "kg")
27 }
```

PS E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji> go run "E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 9\Unguided\Unguided-3\soal-3.go"

• Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 3.4
Masukan berat balita ke-2: 5
Masukan berat balita ke-3: 2.3
Masukan berat balita ke-4: 6
Berat balita minimum: 2.30 kg
Berat balita maksimum: 5.00 kg
• Rata-rata berat balita: 4.17

PS E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji> go run "E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 9\Unguided\Unguided-3\soal-3.go"

• Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 3.2
Masukan berat balita ke-3: 4.5
Masukan berat balita ke-4: 9.4
Berat balita minimum: 3.20 kg
Berat balita maksimum: 9.40 kg
• Rata-rata berat balita: 5.60

PS E:\Code\Kuliah\ALPRO2\109082500005_wahid-Satrio-Aji>

Penjelasan :

Pada tahap awal, program mendefinisikan sebuah tipe data bentukan baru khusus untuk menampung array float64 berkapasitas seratus elemen, yang bertujuan membuat penulisan parameter di dalam fungsi menjadi lebih ringkas dan mudah digunakan. Meskipun fungsi di bahasa Go secara bawaan menerapkan mekanisme pass-by-value yang hanya mengirimkan salinan data, program ini menggantinya pada fungsi pencarian nilai ekstrem dengan mendeklarasikan parameternya sebagai pointer. Hal ini memungkinkan terjadinya proses dereferencing, di mana fungsi tersebut tidak sekadar memanipulasi variabel lokalnya saja, melainkan langsung menelusuri alamat memori untuk mengubah nilai asli dari variabel minimum dan maksimum yang berada di fungsi utama. Sebaliknya, fungsi penghitung rata-rata diimplementasikan sebagai fungsi pass-by-value standar yang murni beroperasi secara independent, ia menerima salinan array, mengakumulasikan total nilainya secara internal, dan langsung mengembalikan hasil pembagian tersebut sebagai output akhir untuk dicetak oleh program utama.

D. Kesimpulan

Berdasarkan praktikum, dapat disimpulkan bahwa pencarian nilai ekstrem (maksimum dan minimum) serta perhitungan rata-rata dapat diimplementasikan secara efisien menggunakan bahasa pemrograman Go. Algoritma Linear Search terbukti menjadi metode yang sederhana namun sangat tangguh untuk mencari nilai tertentu pada kumpulan data yang tidak memiliki pola urutan. Dengan menjadikan elemen pertama sebagai patokan awal, program terhindar dari kesalahan logika atau bug yang mungkin terjadi jika menggunakan angka sembarang sebagai inisialisasi awal.

Praktikum ini juga menyoroti pentingnya pemilihan struktur data yang tepat. Penggunaan array statis sangat cocok untuk alokasi memori yang sudah diketahui batas maksimalnya, sementara slice memberikan fleksibilitas tinggi untuk menampung hasil perhitungan yang dinamis. Lebih jauh lagi, penggunaan struct membuktikan bahwa algoritma dasar ini tidak hanya terbatas pada deretan angka sederhana, melainkan dapat diskalakan untuk memproses objek data dunia nyata yang kompleks, seperti data akademik mahasiswa.

Selain itu, pembagian kode ke dalam beberapa fungsi spesifik sangat mempermudah proses identifikasi masalah (debugging) dan perawatan kode. Praktikum ini juga berhasil mempraktikkan konsep manipulasi memori tingkat lanjut melalui penggunaan pointer, yang memungkinkan sebuah subprogram untuk memperbarui nilai dari variabel minimum dan maksimum secara langsung di memori utama. Secara keseluruhan, pemahaman terhadap perulangan, percabangan, struktur data, dan manajemen memori ini merupakan fondasi yang krusial untuk membangun program pemrosesan data yang skalabel dan efisien.